Lab Assignment 8: Data Management Using `pandas`, Part 1

DS 6001

Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

Problem 0

Import the following packages:

In [32]:
```
import numpy as np
import pandas as pd
import sidetable
import sqlite3
```

Problem 1

There are a lot of resources on the internet on the general topic of Python for Data Science, and many of these websites publish lists of the most important Python packages for data science. There are definite disagreements further down on these lists, but the top spots always include numpy and pandas. We've used pandas in every single one of the labs in this course so far, and modules 8 and 9 are entirely about the process of data wrangling and manipulation using pandas.

But, there is a package that competes directly with pandas, called polars. This package has its fans and adherents. But the tenor and tone of the debate over which package to use is more adversarial and heated than I would expect. For many, it is not enough to use polars when it makes sense as part of a larger data science toolkit: polars needs to completely defeat and replace pandas, and send pandas to that great software recycling bin in the sky.

As a data professional, you will absolutely be drawn into a conversation at some point that's in reality a small battle in the great pandas vs. polars war. Maybe polars really is the inevitable replacement for pandas, maybe not. But you should know about these options to form a solid opinion so that you can hold your ground in one of these

conversations until civility can be reestablished and everyone can just talk about sports or gossip again.

Part a

The polars website contains a page that is directed at pandas users. Please read this page, and find two points made on this page that seem appealing or interesting to you, and describe those points here. [4 points]

The two things I find interesting about Polars are lazy evaluation and parallel processing. lazy evaluation allows Polars to wait until the results are actually needed before running operations. I find this useful because it can save time by avoiding unnecessary steps, especially when working on long tasks. The other thing I find interesting is the ability to use multiple CPU cores at the same time, since this allows tasks to be split across different cores, making the process faster and more efficient.

Part b

So, here's the thing. When there's contention between a new and an old way of doing something, the voices on the side of the new way will usually be louder, though not necessarily greater in number. "This New Way Will Change Everything" makes a more clickable blog title than "There's Nothing Wrong With the Way We Do Things Now." To see the argument for pandas over polars, look for analyses that aim for an objective comparison. Read this blog post by Josep Ferrer, or find a different post or analysis that defends pandas, and in a one or two of your own sentences, describe the best argument for pandas. [4 points]

I think the best argument for pandas is its compatibility, since it works well with many other Python libraries. This makes it easier to use different tools together.

Part c

Do a search on a social media platform of your choice, and find an instance of somebody commenting on the polars vs. pandas debate. Copy and paste some text from this post that your find compelling or interesting, along with the link. Also, do you agree with this opinion, or not, and why? [4 points]

I agree that it would be nice to avoid the confusion with reset_index and inplace in pandas. As for handling date and time data, I can't really say much since I haven't used Polars enough to compare them. However, I do agree that having a simpler and more consistent way of doing things would be helpful.

link: https://www.reddit.com/r/Python/comments/1jg402b/polars_vs_pandas/

Comment: I "grew up" on Pandas, but moved to Polars. No more "reset_index" and "inplace" confusion. Feels like there's only one right way to do it in Polars, but so much bloat in Pandas API.

I do like Pandas when it comes to certain things where there is an obvious index like time signals. But Polars seems to handle date time much better.

When it comes to filtering and queries, I like Polars.

In both, I've made several df and series "helper" attributes to clean up the syntax.

Part d

As someone who either currently, or who will soon be a data professional, you will be a member of a community of similar professionals. We learn a great deal from our peers in these kinds of communities of practice. Have a conversation with someone who is a data professional, whether that is someone you work with, or someone you know, or someone that someone you know knows, and ask them about their thoughts on polars vs pandas. What are their thoughts on this? [4 points]

He thinks pandas is still a better choice for most people because it is widely used and works well with other Python libraries. However, he also thinks Polars is a good option for working with large datasets because it can be faster and more efficient.

Part e

Based on your reading and thoughts from parts (a) through (d), do you think you will try to learn how to use polars, or will you stick with pandas, or try using both? (No wrong answers, just curious about where you land on this.)

By the way: We are going to continue to use pandas in this course, but if you would like to train on polars, one good option is this course on Datacamp. You should have free access to Datacamp from UVA, but if not, please let me know so we can get you access. [4 points]

In my case, I haven't worked with pandas enough to be an expert or fully understand its limitations, but I have felt confident using it so far. However, I think learning another library like Polars could be helpful, and if I start seeing Polars mentioned more often in job postings, I would definitely give it a try.

Problem 2

Please use pandas, not polars, for this problem.

For this problem, you will be working with the 2017 Workplace Health in America survey which was conducted by the Centers for Disease Control and Prevention. According to the survey's guidance document:

> The Workplace Health in America (WHA) Survey gathered information from a cross-sectional, nationally representative sample of US worksites. The sample was drawn from the Dun & Bradstreet (D&B) database of all private and public employers in the United States with at least 10 employees. Like previous national surveys, the worksite served as the sampling unit rather than the companies or firms to which the worksites belonged. Worksites were selected using a stratified simple random sample (SRS) design, where the primary strata were ten multi-state regions defined by the Centers for Disease Control and Prevention (CDC), plus an additional stratum containing all hospital worksites.

The data contain over 300 features that report the industry and type of company where the respondents are employed, what kind of health insurance and other health programs are offered, and other characteristics of the workplaces including whether employees are allowed to work from home and the gender and age makeup of the workforce. The data are full of interesting information, but in order to make use of the data a great deal of data manipulation is required first.

Part a

The raw data are stored in an ASCII file on the 2017 Workplace Health in America survey homepage. Load the raw data directly into Python without downloading the data onto your harddrive. Then use the `.info(verbose=True)` method on the dataframe to show summary information about the data. Based on what you see (it's NOT necessary to usE any other code) answer the following questions:

- How many rows?
- How many columns?
- How many columns have a float data type (numeric with decimals) and how many have an object data type (text or strings)?

[4 points]

In [33]:
```
import pandas as pd

url = "https://www.cdc.gov/workplace-health-promotion/media/files/2024/06/wh

workplace_health_df = pd.read_csv(url, sep="~", low_memory=False)
workplace_health_df.info(verbose=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2843 entries, 0 to 2842
Data columns (total 301 columns):
 #    Column                Dtype
---   ------                -----
 0    OC1                   object
 1    OC3                   float64
 2    HI1                   float64
 3    HI2                   float64
 4    HI3                   float64
 5    HI4                   float64
 6    HRA1                  float64
 7    HRA1A                 float64
 8    HRA1B                 float64
 9    HRA1E                 float64
 10   CP1                   float64
 11   CP2                   float64
 12   CP3                   float64
 13   CP4                   float64
 14   CP5                   float64
 15   HP1                   float64
 16   HP2                   float64
 17   HP3                   float64
 18   HP4                   float64
 19   HP5                   float64
 20   HP5A                  float64
 21   HP6                   float64
 22   HP7A                  float64
 23   HP7B                  float64
 24   HP7C                  float64
 25   HP7D                  float64
 26   HP7D1                 float64
 27   HP7D2                 float64
 28   HP7D3                 float64
 29   HP7E                  float64
 30   HP7E1                 float64
 31   HP7F                  float64
 32   HP7F1                 float64
 33   HP7F2                 float64
 34   HP7F3                 float64
 35   HP7F4                 float64
 36   HP7F5                 float64
 37   HP7F6                 float64
 38   HP7F7                 float64
 39   HP7F8                 float64
 40   HP7F9                 float64
 41   HP7F10                float64
 42   HP7F11                float64
 43   HP7G                  float64
 44   HP8                   float64
 45   HPR1_1                float64
 46   HPR1_1A               float64
 47   HPR1_1B               float64
 48   HPR1_1C               float64
 49   HPR1_2A               float64
 50   HPR1_2B               float64
```

```
51   HPR1_2C              float64
52   HPR1_2D              float64
53   HPR1_2E              float64
54   HPR1_2F              float64
55   HPR1_2G              float64
56   HPR1_2H              float64
57   HPR1_2I              float64
58   HPR1_2J              float64
59   HPR1_2K              float64
60   HPR2_1               float64
61   HPR2_1A              float64
62   HPR2_1B              float64
63   HPR2_1C              float64
64   HPR2_2A              float64
65   HPR2_2B              float64
66   HPR2_2C              float64
67   HPR2_2D              float64
68   HPR2_3A              float64
69   HPR2_3B              float64
70   HPR2_3C              float64
71   HPR2_3D              float64
72   HPR2_4A              float64
73   HPR2_4B              float64
74   HPR2_4C              float64
75   HPR2_4D              float64
76   HPR2_4E              float64
77   HPR3_1               float64
78   HPR3_1A              float64
79   HPR3_1B              float64
80   HPR3_1C              float64
81   HPR3_2               float64
82   HPR4_1               float64
83   HPR4_1A              float64
84   HPR4_1B              float64
85   HPR4_1C              float64
86   HPR4_2A              float64
87   HPR4_2B              float64
88   HPR4_2C              float64
89   HPR4_2D              float64
90   HPR4_2E              float64
91   HPR4_2F              float64
92   HPR4_2G              float64
93   HPR4_2G1             float64
94   HPR4_2G2             float64
95   HPR4_2G3             float64
96   HPR4_2G4             float64
97   HPR4_2G5             float64
98   HPR4_2G6             float64
99   HPR4_2G7             float64
100  HPR5_1               float64
101  HPR5_1A              float64
102  HPR5_1B              float64
103  HPR6_1               float64
104  HPR6_1A              float64
105  HPR6_1B              float64
106  HPR6_1C              float64
```

```
107  HPR7_1              float64
108  HPR7_1A             float64
109  HPR7_1B             float64
110  HPR7_1C             float64
111  HPR8_1              float64
112  HPR8_1A             float64
113  HPR8_1B             float64
114  HPR8_1C             float64
115  HPR9_1              float64
116  HPR9_1A             float64
117  HPR9_1B             float64
118  HPR9_1C             float64
119  HS11                float64
120  HS11_2              float64
121  HS11A               float64
122  HS12                float64
123  HS12_2              float64
124  HS13                float64
125  HS13_2              float64
126  HS14                float64
127  HS14_2              float64
128  HS15                float64
129  HS15_2              float64
130  HS16                float64
131  HS16_2              float64
132  HS17                float64
133  HS17_2              float64
134  HS18                float64
135  HS18_2              float64
136  HS19                float64
137  HS19_2              float64
138  HS2A                float64
139  HS2B                float64
140  HS3                 float64
141  DM11M1              float64
142  DM11M2              float64
143  DM11M3              float64
144  DM12M1              float64
145  DM12M2              float64
146  DM12M3              float64
147  DM13M1              float64
148  DM13M2              float64
149  DM13M3              float64
150  DM14M1              float64
151  DM14M2              float64
152  DM14M3              float64
153  DM15M1              float64
154  DM15M2              float64
155  DM15M3              float64
156  DM16M1              float64
157  DM16M2              float64
158  DM16M3              float64
159  DM17M1              float64
160  DM17M2              float64
161  DM17M3              float64
162  DM18M1              float64
```

```
163  DM18M2              float64
164  DM18M3              float64
165  DM19M1              float64
166  DM19M2              float64
167  DM19M3              float64
168  DM20M1              float64
169  DM20M2              float64
170  DM20M3              float64
171  DM2A                float64
172  DM2B                float64
173  DM3                 float64
174  KP2                 float64
175  KP3_1               float64
176  KP3_2               float64
177  KP3_3               float64
178  KP4                 float64
179  KP5A                float64
180  KP5C                float64
181  KP5E                float64
182  KP5F                float64
183  KP5G                float64
184  KP5H                float64
185  KP5J                float64
186  KP5J_01             object
187  WL1                 float64
188  WL2                 float64
189  WL3M1               float64
190  WL3M2               float64
191  WL3M3               float64
192  WL3M4               float64
193  WL3M5               float64
194  WL5                 float64
195  WL6                 float64
196  WL7                 float64
197  WL8                 float64
198  WL9                 float64
199  WL11                float64
200  WL12                float64
201  WL14                float64
202  WL15                float64
203  B1_1                float64
204  B1_2                float64
205  B1_3                float64
206  B1_4                float64
207  B1_5                float64
208  B1_6                float64
209  B1_7                float64
210  B1_8                float64
211  B1_9                float64
212  B1_10               float64
213  B1_11               float64
214  B1_12               float64
215  OSH1                float64
216  OSH2                float64
217  OSH3                float64
218  OSH4                float64
```

```
219  OSH5                float64
220  OSH6                float64
221  OSH7_1              float64
222  OSH7_2              float64
223  OSH7_3              float64
224  E1M1                float64
225  E1M2                float64
226  E1M3                float64
227  E1M4                float64
228  E1M5                float64
229  E1M6                float64
230  E1M7                float64
231  E1M8                float64
232  E1M9                float64
233  E2                  float64
234  WD1_1               float64
235  WD1_2               float64
236  WD2                 float64
237  WD3                 float64
238  WD4                 float64
239  WD5                 float64
240  WD6                 float64
241  WD7                 float64
242  HPR5_2A_S           float64
243  HPR5_2B_S           float64
244  HPR5_2C_S           float64
245  HPR5_2D_S           float64
246  HPR5_2E_S           float64
247  WL1_1_S             float64
248  WL1_2_S             float64
249  WL1_3_S             float64
250  WL1_4_S             float64
251  WL1_5_S             float64
252  HPR9_2A_S           float64
253  HPR9_2B_S           float64
254  HPR9_2C_S           float64
255  HPR9_2D_S           float64
256  HPR9_2E_S           float64
257  HPR9_2F_S           float64
258  HPR8_2A_S           float64
259  HPR8_2B_S           float64
260  HPR8_2C_S           float64
261  HPR8_2D_S           float64
262  HPR8_2E_S           float64
263  HPR7_2A_S           float64
264  HPR7_2C_S           float64
265  HPR7_2D_S           float64
266  HPR7_2E_S           float64
267  HPR7_2F_S           float64
268  HPR6_2A_S           float64
269  HPR6_2B_S           float64
270  HPR6_2C_S           float64
271  HPR6_2D_S           float64
272  HPR6_2E_S           float64
273  OSH7_1_S            float64
274  OSH7_2_S            float64
```

```
 275  OSH7_4_S             float64
 276  OSH7_6_S             float64
 277  OSH8_S               float64
 278  OSH81A_S             float64
 279  OSH81B_S             float64
 280  OSH81C_S             float64
 281  KP1A_S               float64
 282  KP1B_S               float64
 283  KP1C_S               float64
 284  KP1D_S               float64
 285  KP1E_S               float64
 286  KP1F_S               float64
 287  KP1G_S               float64
 288  KP1G_S_01            object
 289  HOSPITAL             float64
 290  OC1_07               object
 291  WL3_05               object
 292  E1_09                object
 293  Suppquex             float64
 294  Id                   float64
 295  Region               float64
 296  CDC_Region           float64
 297  Industry             float64
 298  Size                 float64
 299  Varstrata            float64
 300  Finalwt_worksite,,,,  object
dtypes: float64(294), object(7)
memory usage: 6.5+ MB
```

2,843 rows

301 columns

294 float64 columns

7 object columns

Part b

Create a new variable in Python's memory to store a working version of the data. In the working version, delete all of the columns except for the following:

- `Industry`: 7 Industry Categories with NAICS codes
- `Size`: 8 Employee Size Categories
- `OC3` Is your organization for profit, non-profit, government?
- `HI1` In general, do you offer full, partial or no payment of premiums for personal health insurance for full-time employees?
- `HI2` Over the past 12 months, were full-time employees asked to pay a larger proportion, smaller proportion or the same proportion of personal health insurance

premiums?

- `HI3`: Does your organization offer personal health insurance for your part-time employees?
- `CP1`: Are there health education programs, which focus on skill development and lifestyle behavior change along with information dissemination and awareness building?
- `WL6`: Allow employees to work from home?
- Every column that begins `WD`, expressing the percentage of employees that have certain characteristics at the firm

[4 points]

In [34]:
```
columns_to_keep = [ "Industry", "Size", "OC3", "HI1", "HI2", "HI3", "CP1", "
wd_columns = [col for col in workplace_health_df.columns if col.startswith("

workplace_health_df = workplace_health_df.loc[:, columns_to_keep + wd_columr
workplace_health_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2843 entries, 0 to 2842
Data columns (total 16 columns):
 #   Column    Non-Null Count  Dtype  
---  ------    --------------  -----  
 0   Industry  2842 non-null   float64
 1   Size      2842 non-null   float64
 2   OC3       2842 non-null   float64
 3   HI1       2842 non-null   float64
 4   HI2       2842 non-null   float64
 5   HI3       2842 non-null   float64
 6   CP1       2842 non-null   float64
 7   WL6       2842 non-null   float64
 8   WD1_1     2842 non-null   float64
 9   WD1_2     2842 non-null   float64
 10  WD2       2842 non-null   float64
 11  WD3       2842 non-null   float64
 12  WD4       2842 non-null   float64
 13  WD5       2842 non-null   float64
 14  WD6       2842 non-null   float64
 15  WD7       2842 non-null   float64
dtypes: float64(16)
memory usage: 355.5 KB
```

Part c

Column names like 'OC3' and 'CP1' do not convey the meaning or intuition of a feature, so let's choose better names. The codebook for the WHA data contain short descriptions of the meaning of each of the columns in the data. Use these descriptions

to decide on better and more intuitive names for the columns in the working version of the data, and rename the columns accordingly. [4 points]

In [35]:
```
column_names = {
    "Industry": "industry",
    "Size": "employee_size",
    "OC3": "organization_type",
    "HI1": "full_time_insurance_coverage",
    "HI2": "insurance_premium_change",
    "HI3": "part_time_health_insurance",
    "CP1": "health_education_programs",
    "WL6": "work_from_home"
}

workplace_health_df = workplace_health_df.rename(columns=column_names)
```

Part d

Throughout the data, numeric values are used to represent categories. For example, for the feature that asks whether a workplace allows employees to work from home, 1 means yes and 2 means no. But it is more intuitive, less error prone, and easier for data visualization and other analyses if we replace these numbers with text labels that tell us explicitly what the categories represent. The following questions ask you to replace numeric values with text labels for these categorical features.

Step i

The industry column is currently coded numerically:

In [36]:
```
whpps['Industry'].value_counts()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[36], line 1
----> 1 whpps['Industry'].value_counts()

NameError: name 'whpps' is not defined
```

According to the codebook, these values represent:

- 1 = Industry Category 1: NAICS Sectors: 11, 21, 22, 23, 31-33
- 2 = Industry Category 2: NAICS Sectors: 42, 44-45, 48-49
- 3 = Industry Category 3: NAICS Sectors: 71, 72, 81
- 4 = Industry Category 4: NAICS Sectors: 51, 52, 53, 54, 55, 56
- 5 = Industry Category 5: NAICS Sectors: 61, 62 (excluding hospital worksites)
- 6 = Industry Category 6: NAICS Sectors: 92
- 7 = Industry Category 7: Hospital worksites (NAICS6 = 622110, 622210, 622310)

NAICS stands for North American Industry Classification System. According to their website, their industry coding system

> is a standardized system used to classify businesses based on their primary activities. This NAICS Code classification helps in the collection, analysis, and publication of statistical data related to the U.S. economy. Accurate NAICS Information is crucial for businesses because they affect eligibility for government contracts, loans, and tax benefits, and they facilitate better communication with government agencies.

Use the dictionary of NAICS industrial codes to lookup the NAICS codes that each category of the Industry feature represents, then place descriptive labels on the categories of the industry column in the working data. This is a bit of a pain because you need to find a succinct but descriptive way to label each category. For example, Industry=1 represents Agriculture, Forestry, Fishing and Hunting (NAICS code 11), Mining (NAICS code 21), Utilities (NAICS code 22), Construction (NAICS code 23), and Manufacturing (NAICS code 31-33). You can use the label 'Agriculture, Construction, Manufacturing' if you want, or something else. Do your best.

[4 points]

In [37]:
```
industry_labels = {
    1: "Agriculture, Construction, Manufacturing",
    2: "Wholesale, Retail, Transportation",
    3: "Arts, Entertainment, Hospitality, Services",
    4: "Information, Finance, Professional Services",
    5: "Education, Healthcare (Non-Hospital)",
    6: "Public Administration",
    7: "Hospitals"
}

workplace_health_df["industry"] = (
    workplace_health_df["industry"].replace(industry_labels)
)

workplace_health_df["industry"].value_counts()
```

Out[37]:
```
industry
Education, Healthcare (Non-Hospital)           551
Agriculture, Construction, Manufacturing       525
Arts, Entertainment, Hospitality, Services     433
Information, Finance, Professional Services    429
Hospitals                                      338
Wholesale, Retail, Transportation              311
Public Administration                          255
Name: count, dtype: int64
```

Step ii

Using the codebook, recode the "size" feature to have three categories: "Small" for workplaces with fewer than 100 employees, "Medium" for workplaces with at least 100 but fewer than 500 employees, and "Large" for companies with at least 500 employees. [Note: Python dataframes have an attribute `.size` that reports the space the dataframe takes up in memory. Don't confuse this attribute with the column named "Size" in the raw data.] [4 points]

In [38]:
```
size_labels = {
    1: "Small",
    2: "Small",
    3: "Small",
    4: "Medium",
    5: "Medium",
    6: "Large",
    7: "Large",
    8: "Large"
}

workplace_health_df["employee_size"] = (
    workplace_health_df["employee_size"].replace(size_labels)
)

workplace_health_df["employee_size"].value_counts()
```

Out[38]:
```
employee_size
Small     2195
Medium     393
Large      254
Name: count, dtype: int64
```

Step iii

Use the codebook to write accurate and descriptive labels for each category for each categorical column in the working data. Then apply all of these labels to the data at once. Code "Legitimate Skip", "Don't know", "Refused", and "Blank" as `np.nan` missing values. [8 points]

In [39]:
```
import numpy as np

category_labels = {
    "organization_type": {
        1: "For-profit public",
        2: "For-profit private",
        3: "Non-profit",
        4: "State/local government",
        5: "Federal government",
        6: "Other"
    },
    "full_time_insurance_coverage": {
        1: "Full coverage",
        2: "Partial coverage",
        3: "No coverage"
```

```
    },
    "insurance_premium_change": {
        1: "Larger",
        2: "Smaller",
        3: "About the same"
    },
    "part_time_health_insurance": {
        1: "Yes",
        2: "No"
    },
    "health_education_programs": {
        1: "Yes",
        2: "No"
    },
    "work_from_home": {
        1: "Yes",
        2: "No"
    }
}

workplace_health_df = workplace_health_df.replace(category_labels)

missing_codes = [96, 97, 98, 99]
categorical_columns = list(category_labels)

workplace_health_df[categorical_columns] = (
    workplace_health_df[categorical_columns].replace(missing_codes, np.nan)
)

workplace_health_df[wd_columns] = (
    workplace_health_df[wd_columns].replace(
        [997, 998, 999], np.nan
    )
)
```

In [40]:
```
workplace_health_df.head().T
```

Out[40]:

	0	1	2	3	4
industry	Hospitals	Hospitals	Hospitals	Hospitals	Hospitals
employee_size	Large	Large	Large	Medium	Medium
organization_type	Non-profit	Non-profit	Non-profit	For-profit private	Non-profit
full_time_insurance_coverage	Partial coverage	Partial coverage	Full coverage	Full coverage	Full coverage
insurance_premium_change	Larger	About the same	About the same	Smaller	About the same
part_time_health_insurance	No	Yes	Yes	Yes	Yes
health_education_programs	Yes	Yes	Yes	No	Yes
work_from_home	Yes	Yes	Yes	No	Yes
WD1_1	25.0	NaN	35.0	50.0	50.0
WD1_2	20.0	NaN	4.0	15.0	40.0
WD2	85.0	90.0	NaN	50.0	60.0
WD3	60.0	90.0	NaN	85.0	60.0
WD4	40.0	NaN	40.0	75.0	40.0
WD5	15.0	NaN	15.0	0.0	30.0
WD6	0.0	0.0	NaN	0.0	0.0
WD7	22.0	NaN	NaN	NaN	28.0

Step iv

The features that measure the percent of the workforce with a particular characteristic use the codes 997, 998, and 999 to represent "Don't know", "Refusal", and "Blank/Invalid" respectively. If we don't change these, all numeric results that use these features will be wrong. Replace these values with missing values for all of the percentage features at the same time. [4 points]

In [41]:
```
workplace_health_df.isna().sum().sum()
```

Out[41]:
```
np.int64(8799)
```

In [42]:
```
wd_columns = [
    col for col in workplace_health_df.columns
    if col.startswith("WD")
]

workplace_health_df[wd_columns] = (
    workplace_health_df[wd_columns].replace(
        [997, 998, 999], np.nan
```

```
    )
)
```

In [43]:
```
workplace_health_df.isna().sum().sum()
```

Out[43]: np.int64(8799)

Part e

As you proceed through a data wrangling process, viewing the data at each step is important to determine whether your code has worked correctly and to see the next problems that need to be solved. Sorting the data is one of the most important and useful ways to look at different parts of the data that are not displayed by default within the `.head()` of a dataframe.

Sort the working data by industry in ascending alphabetical order. Within industry categories, sort the rows by size in ascending alphabetical order. Within groups with the same industry and size, sort by percent of the workforce that is under 30 in descending numeric order. [4 points]

In [46]:
```
workplace_health_df = workplace_health_df.sort_values(
    by=["industry", "employee_size", "WD1_1"],
    ascending=[True, True, False]
)

workplace_health_df.head().T
```

Out[46]:

	1732	1476	1477	70
industry	Agriculture, Construction, Manufacturing	Agriculture, Construction, Manufacturing	Agriculture, Construction, Manufacturing	Agriculture Constructior Manufacturin
employee_size	Large	Large	Large	Larg
organization_type	For-profit private	For-profit private	For-profit private	For-prof privat
full_time_insurance_coverage	Partial coverage	Partial coverage	Partial coverage	Full coverag
insurance_premium_change	About the same	About the same	Smaller	About th sam
part_time_health_insurance	No	No	No	N
health_education_programs	Yes	Yes	Yes	Ye
work_from_home	No	No	Yes	Ye
WD1_1	50.0	40.0	25.0	20.
WD1_2	10.0	10.0	15.0	15.
WD2	50.0	30.0	20.0	17.
WD3	75.0	60.0	60.0	62.
WD4	10.0	30.0	10.0	10.
WD5	0.0	5.0	2.0	5.
WD6	0.0	0.0	60.0	0.
WD7	75.0	10.0	5.0	11.

Part f

Sometimes important features have missing values for one reason or another, and depending on our purposes, we may not be able to use rows with these missing values. In those situations, it is often easier to delete the rows with missing values.

There is one row in the working data that has a missing value for industry. Delete this row. [4 points]

In [47]:
```
workplace_health_df = workplace_health_df.dropna(
    subset=["industry"]
)

workplace_health_df["industry"].isna().sum()
```

Out[47]:
```
np.int64(0)
```

Part g

Sometimes we need to take a continuous-valued feature and turn it into a categorical one for some analysis.

Create a new feature named `gender_balance` that has three categories: "Mostly men" for workplaces with between 0% and 35% female employees, "Balanced" for workplaces with more than 35% and at most 65% female employees, and "Mostly women" for workplaces with more than 65% female employees. [4 points]

In [48]:
```
workplace_health_df["gender_balance"] = pd.cut(
    workplace_health_df["WD2"],
    bins=[0, 35, 65, 100],
    labels=["Mostly men", "Balanced", "Mostly women"],
    include_lowest=True
)

workplace_health_df["gender_balance"].value_counts()
```

Out[48]:
```
gender_balance
Mostly men      733
Mostly women    727
Balanced        587
Name: count, dtype: int64
```

Part h

Currently all of the categorical features in the data are stored as "object" data types, which is the same data type as strings. But there are additional methods and display properties that are available if we instead convert these features to a "category" data type.

Change the data type of all categorical features in the working data from "object" to "category". [4 points]

In [49]:
```
categorical_columns = workplace_health_df.select_dtypes(
    include=["object"]
).columns

workplace_health_df[categorical_columns] = (
    workplace_health_df[categorical_columns].astype("category")
)

workplace_health_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 2842 entries, 1732 to 2631
Data columns (total 17 columns):
 #   Column                         Non-Null Count  Dtype
---  ------                         --------------  -----
 0   industry                       2842 non-null   category
 1   employee_size                  2842 non-null   category
 2   organization_type              2613 non-null   category
 3   full_time_insurance_coverage   2763 non-null   category
 4   insurance_premium_change       2249 non-null   category
 5   part_time_health_insurance     2682 non-null   category
 6   health_education_programs      2668 non-null   category
 7   work_from_home                 2549 non-null   category
 8   WD1_1                          1874 non-null   float64
 9   WD1_2                          1901 non-null   float64
 10  WD2                            2047 non-null   float64
 11  WD3                            1907 non-null   float64
 12  WD4                            1986 non-null   float64
 13  WD5                            2058 non-null   float64
 14  WD6                            2151 non-null   float64
 15  WD7                            1557 non-null   float64
 16  gender_balance                 2047 non-null   category
dtypes: category(9), float64(8)
memory usage: 226.2 KB
```

Problem 3

Just like SQL is a mechanism for deriving answers from the information stored within a database, pandas can be a tool for getting answers to questions using a dataframe.

The following questions ask you to perform some simple analyses using data wrangling alone.

Part a

I am looking for a job with a small workplace that lets me work from home, but I am worried about whether the job will provide me with health insurance.

Filter the data to only those rows that represent small workplaces that allow employees to work from home. Then report how many of these workplaces offer full insurance, partial insurance, and no insurance. Report the percent, cumulative count, and cumulative percent in addition to the counts.

This is much easier if you use the `sidetable` package. Any answer that does not use `sidetable` will receive no credit. [4 points]

In [51]:
```
import sidetable

small_remote_workplaces = workplace_health_df.loc[
    (workplace_health_df["employee_size"] == "Small") &
    (workplace_health_df["work_from_home"] == "Yes")
```

```
]

small_remote_workplaces.stb.freq(
    ["full_time_insurance_coverage"]
)
```

```
/Users/jonathanorellana/miniconda3/envs/ds6001/lib/python3.13/site-packages/
sidetable/sidetable.py:103: FutureWarning: The default of observed=False is
deprecated and will be changed to True in a future version of pandas. Pass o
bserved=False to retain current behavior or observed=True to adopt the futur
e default and silence this warning.
  group_data = self._obj.groupby(cols).size().reset_index(
```

Out[51]:

	full_time_insurance_coverage	count	percent	cumulative_count	cumulative_perc
0	Full coverage	324	46.285714	324	46.285
1	Partial coverage	310	44.285714	634	90.571
2	No coverage	66	9.428571	700	100.000

Part b

Anything that can be done in SQL can be done with `pandas`. The next several questions ask you to write `pandas` code to match a given SQL query.

But first, to check that the SQL query and `pandas` code yield the same result, create a new database using the `sqlite3` package and input the cleaned WHA data as a table in this database. [4 points]

In [52]:
```
import sqlite3

conn = sqlite3.connect("workplace_health.db")

workplace_health_df.to_sql(
    "workplace_health",
    conn,
    if_exists="replace",
    index=False
)
```

Out[52]: 2842

Part c

Some hospital workplaces lowered their insurance premiums. I want to know whether these hospitals were large, medium, or small employers, for profit or non-profit, whether they offer full or partial insurance, and whether they have part-time employees. Using SQL, I can get an answer to these questions with this query:

```
SELECT size,
    type,
```

```
        premiums AS insurance,
        parttime
    FROM whpps
    WHERE industry = 'Hospitals'
        AND premium_change='Smaller'
```

Write pandas code that replicates the results of this SQL query. Your feature names might be different from the ones listed in the query, depending on the column names you chose earlier. [8 points]

In [56]:
```
# To Check the results of the SQL query
sql_query = """
SELECT
    employee_size,
    organization_type,
    full_time_insurance_coverage,
    part_time_health_insurance
FROM workplace_health
WHERE industry = 'Hospitals'
    AND insurance_premium_change = 'Smaller'
"""

sql_result = pd.read_sql_query(sql_query, conn)
sql_result
```

Out[56]:

	employee_size	organization_type	full_time_insurance_coverage	part_time_health_
0	Large	Non-profit	Partial coverage	
1	Large	Non-profit	Partial coverage	
2	Large	Non-profit	Partial coverage	
3	Medium	For-profit private	Full coverage	
4	Medium	Non-profit	Full coverage	
5	Medium	Non-profit	Full coverage	
6	Medium	None	Partial coverage	
7	Medium	Non-profit	Partial coverage	
8	Medium	Non-profit	Full coverage	
9	Medium	Non-profit	Partial coverage	
10	Small	Non-profit	Full coverage	

In [54]:
```
hospitals = workplace_health_df.loc[
    (workplace_health_df["industry"] == "Hospitals") &
    (workplace_health_df["insurance_premium_change"] == "Smaller"),
    [
        "employee_size",
        "organization_type",
        "full_time_insurance_coverage",
```

```
        "part_time_health_insurance"
    ]
]

hospitals
```

Out[54]:

	employee_size	organization_type	full_time_insurance_coverage	part_time_healt
187	Large	Non-profit	Partial coverage	
97	Large	Non-profit	Partial coverage	
214	Large	Non-profit	Partial coverage	
3	Medium	For-profit private	Full coverage	
75	Medium	Non-profit	Full coverage	
320	Medium	Non-profit	Full coverage	
11	Medium	NaN	Partial coverage	
48	Medium	Non-profit	Partial coverage	
51	Medium	Non-profit	Full coverage	
191	Medium	Non-profit	Partial coverage	
229	Small	Non-profit	Full coverage	

Part d

Are there differences across industry in the percent of the workforce that are women, under 30, or over 60?

Using SQL, I can get an answer to these questions with this query:

```
SELECT industry,
    AVG(percent_female) AS percent_female,
    AVG(percent_under30) AS percent_under30,
    AVG(percent_over60) AS percent_over60
FROM whpps
GROUP BY industry
ORDER BY percent_female DESC
```

Write pandas code that replicates the results of this SQL query. Your feature names might be different from the ones listed in the query, depending on the column names you chose earlier. [8 points]

In [59]:

```
industry_summary = (
    workplace_health_df
    .groupby("industry", observed=True)[["WD2", "WD1_1", "WD1_2"]]
    .mean()
    .sort_values("WD2", ascending=False)
    .reset_index()
```

```
)

industry_summary
```

Out[59]:

	industry	WD2	WD1_1	WD1_2
0	Education, Healthcare (Non-Hospital)	80.657143	25.745665	11.349570
1	Hospitals	76.427027	27.213793	16.489655
2	Arts, Entertainment, Hospitality, Services	53.804416	38.566343	11.544872
3	Information, Finance, Professional Services	50.632184	23.821752	12.465465
4	Public Administration	39.056738	21.015625	15.015385
5	Wholesale, Retail, Transportation	32.657258	29.108696	12.584034
6	Agriculture, Construction, Manufacturing	20.328605	22.257143	10.690355

Part e

Are there differences in whether full, partial, or no health insurance is provided to employees based on whether the workplace is mostly men, mostly women, or is gender balanced?

Using SQL, I can get an answer to this question with this query:

```
SELECT gender_balance,
    premiums,
    COUNT(*)
FROM whpps
GROUP BY gender_balance, premiums
HAVING gender_balance is NOT NULL
    AND premiums is NOT NULL
```

Write pandas code that replicates the results of this SQL query. Your feature names might be different from the ones listed in the query, depending on the column names you chose earlier.

Note: the rows of your pandas output will be sorted in a different order than the rows of the SQL query. That happens due to the default sorting order (SQL sorts alphabetically, pandas sorts according to the order supplied in the `pd.cut()` method you used earlier.) It's a pain to get the sorting to match, so don't! Also, don't worry if the columns have different names. Just confirm the same data is contained in each result. [8 points]

In [62]:

```
# To confirm the same data is contained in each result

sql_query = """
SELECT
    gender_balance,
    full_time_insurance_coverage,
```

```
    COUNT(*) AS count
FROM workplace_health
GROUP BY gender_balance, full_time_insurance_coverage
HAVING gender_balance IS NOT NULL
    AND full_time_insurance_coverage IS NOT NULL
"""

sql_result = pd.read_sql_query(sql_query, conn)
sql_result
```

Out[62]:

	gender_balance	full_time_insurance_coverage	count
0	Balanced	Full coverage	226
1	Balanced	No coverage	77
2	Balanced	Partial coverage	271
3	Mostly men	Full coverage	301
4	Mostly men	No coverage	91
5	Mostly men	Partial coverage	332
6	Mostly women	Full coverage	267
7	Mostly women	No coverage	107
8	Mostly women	Partial coverage	333

In [60]:

```
insurance_summary = (
    workplace_health_df
    .groupby(
        ["gender_balance", "full_time_insurance_coverage"],
        observed=True
    )
    .size()
    .reset_index(name="count")
)

insurance_summary
```

Out[60]:

	gender_balance	full_time_insurance_coverage	count
0	Mostly men	Full coverage	301
1	Mostly men	No coverage	91
2	Mostly men	Partial coverage	332
3	Balanced	Full coverage	226
4	Balanced	No coverage	77
5	Balanced	Partial coverage	271
6	Mostly women	Full coverage	267
7	Mostly women	No coverage	107
8	Mostly women	Partial coverage	333